



# ZYE660 数字衰减器

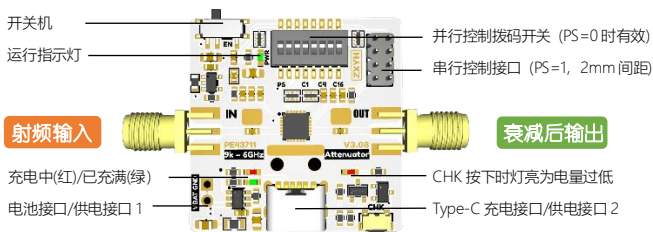
## 使用说明书



## 01/重要注意事项

- 1、 衰减器≠自动功率控制器，不支持指定功率信号输出；
- 2、 ZYE660 电源直供款支持 3.4V-20V 电源供电，充电款支持 3.4V~7V, 请注意区分版本勿上错电压；
- 3、 ZYE660 不支持直流偏置输入，最低工作频率为 9kHz；
- 4、 ZYE660 具有开关机功能，只有 PWR 丝印处的绿色电源指示灯点亮时才开始工作；
- 5、 设定的功率衰减值是相对变化量，且有一定的衰减误差和相位误差，如需精准设定，应以实际测得为准。

## 02/产品概述



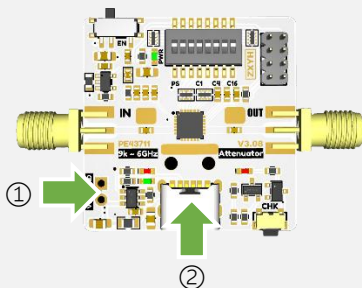
注：电源直供款无充电功能和低电检测功能，供电接口 1 和接口 2 二选一供电！

### ***ZYE660 模块已知被应用的场景：***

无线通信、自动功率控制系统、接收机、发射机、ADC 前端射频信号调节、本振源功率控制、辅助射频调试、功能开发、性能评估等。

## 03/快速开始使用（并行控制）

### 步骤一：接入电源

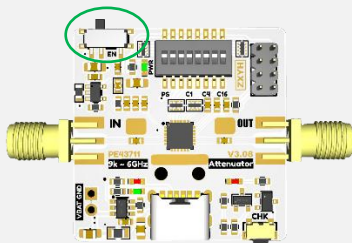


ZYE660-C: ①接入 4.2V 锂离子充电电池,电量不足时使用 Type-C 线接入②进行充电,支持边充电边用(不推荐);

ZYE660-P: ①或②接入直流电源供电, ①接口为 XH2.54, ②接口为 Type-C;

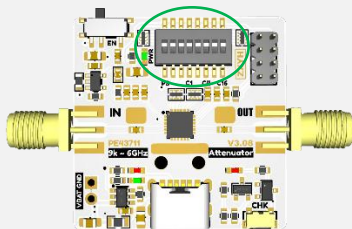
ZYE660-M: 同 ZYE660-C, 已内置电池。

### 步骤二：开机



将右上角的电源开关拨至左侧(如图所示位置), PWR 灯点亮, 衰减器开始工作。

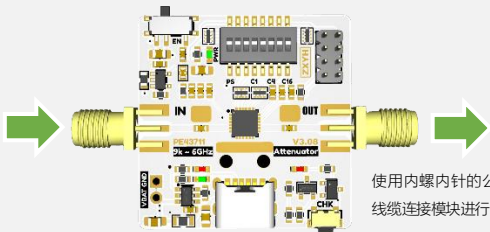
### 步骤三：拨码快速配置衰减值



拨码开关共八位，除第一位为串并控制位以外，后七位都为并行的衰减控制位。第一位往下拨为 0 启用并行控制。

衰减挡位从左到右分别为：  
0.25、0.5、1、2、4、8、16（单位 dB），总衰减量为使能的挡位之和。往上拨为 1 使能开。

### 步骤四：射频连接并使用



使用内螺内针的公头 SMA 线缆连接模块进行使用

## 04/基本参数

产品名称：7 位数字可编程衰减器（主控：PE43711）

型 号：ZYE660

工作频段：9kHz ~ 6GHz

衰减范围：0dB ~ 31.75dB

模块尺寸：33mm x 33mm x 8mm（单模块，不含连接器）

最小步进：0.25dB

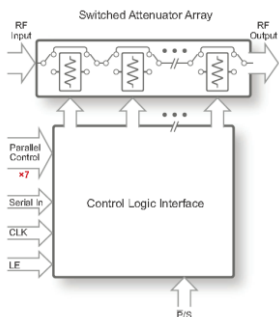
插入损耗：1.5dBm@1GHz

回波损耗：≤-15dB

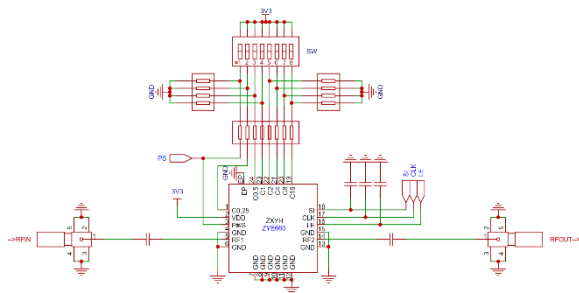
阻 抗：50Ω

工作温度：-20° ~ +50°

## 05/数字衰减器逻辑框图



## 06/模块原理图



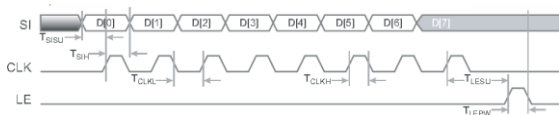
注：原理图仅包含主控部分

## 07/SPI 串行控制介绍

### ● 注意事项

ZYE660 模块可使用 SPI 串行接口通过外部单片机或 USB 转 SPI 模块进行控制，使用前需将拨码开关的第一位（PS）往上拨为 1 启用串行模式。

### ● 控制时序



CLK: 时钟线，一个完整的 SPI 数据包共有 8 个时钟周期；

SI: 数据线，时钟的上升沿衰减器锁存数据线的电平状态，从 Data[0]位开始，它对应的 0.25dB 衰减档位，Data [1]位对应 0.5dB 衰减档位，以此类推共 7bit 位；

LE: 使能位/触发位，将刚收到的 8 个时钟周期衰减配置执行下去。

### ● 串行控制与并行控制的差异

并行控制：一条控制线独立控制一个衰减档，挡位控制支持同时操作且不分先后顺序。优点是无需单独使能，实时性高，执行速度快，但需占用 7 条控制线。

串行控制：挡位控制必须按顺序写入，并且需要单独进行使能，每执行一次衰减变更都需要 9 个步骤（8 个时钟周期+触发）因此速度相对较慢，但只需 3 条控制线。

### ● ZYE660 串行接口引脚定义



## 08/串行控制核心编程逻辑

衰减器采用串行 8 位二进制数据控制，衰减值与数据之间的转换看似复杂，实则只需简单运算即可完成。由于各衰减档以 2 倍关系递增，且最小步进为 0.25 dB，因此只需将目标衰减值乘以 4，即可得到对应的控制数据，转换程序如下：

```
uint8_t att_code = (uint8_t)(att * 4.0);
```

*att*: 需要配置的衰减值      *att\_code*: 转换成串行数据的衰减值

SPI 时序控制本质上可以拆解为 8 位数据的逐位传输循环，以及一次使能信号的电平翻转。使用 HAL 库实现 SPI 串行通信的示例程序如下：

```
void write_ZYE660(uint8_t *att_code)
{
    uint8_t i;
    for (i = 0; i < 8; i++)
    {
        HAL_GPIO_WritePin(ATT_SI_GPIO_Port, ATT_SI_Pin,
(*att_code & 0x01) ? GPIO_PIN_SET : GPIO_PIN_RESET);
        *att_code >>= 1;
        delay_us(1);
        HAL_GPIO_WritePin(ATT_CLK_GPIO_Port, ATT_CLK_Pin,
GPIO_PIN_SET);
        delay_us(2);
        HAL_GPIO_WritePin(ATT_CLK_GPIO_Port, ATT_CLK_Pin,
GPIO_PIN_RESET);
        delay_us(1);
    }
    delay_us(2);
    HAL_GPIO_WritePin(ATT_LE_GPIO_Port, ATT_LE_Pin,
GPIO_PIN_SET);
    delay_us(2);
    HAL_GPIO_WritePin(ATT_LE_GPIO_Port, ATT_LE_Pin,
GPIO_PIN_RESET);
}
```